



SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF MECHANICAL AND AEROSPACE ENGINEERING

Homework 2 — Powered Descent and Landing via Direct Collocation and Successive Convexification

DYNAMICS AND CONTROL OF LAUNCH VEHICLES

Professor:

Prof. A. Zavoli

Student:

Niccolò D'Ambrosio

Contents

1	Introduction	3
1.1	Vehicle Model and Equations of Motion	3
1.2	Boundary Conditions and Mission Parameters	4
1.3	Path Constraints	5
1.4	Optimal Control Problem	5
2	Trapezoidal Direct Collocation	6
2.1	Trapezoidal Quadrature Rule	6
2.2	Defect Constraints	6
2.3	NLP Formulation	7
2.4	KKT Optimality Conditions	7
2.5	Fuel-Optimal Thrust Structure	8
2.6	Discrete Multipliers as Costate Estimates	8
2.7	Sequential Quadratic Programming	9
3	Numerical Implementation	10
3.1	Non-Dimensionalisation	10
3.2	Decision Vector Layout	11
3.3	Initial Guess	12
3.4	Constraints	12
3.5	Solver Settings	12
4	Numerical Results	14
4.1	Trajectory and Glide-Slope Corridor	14
4.2	Thrust Magnitude	15
4.3	Mass and Glide-Slope Angle	16
4.4	Grid Convergence	17
5	Sensitivity Analysis	19
6	Task 2: Zero-Order Hold Discretization	21
6.1	ZOH Transcription for LTV Systems (Appendix A)	21
6.2	Application to the Nonlinear Powered-Descent Problem	22
6.3	Convex-Program Structure of the Inner Step	25
6.4	Lossless Convexification of the Thrust Constraint	26
6.5	Forward-Integration Validation	26
6.6	Results	26
7	Conclusions	33
A	Implementation: Key Code Excerpts	35

1 Introduction

This report presents the solution to Homework 2 of the course Dynamics and Control of Launch Vehicles: the minimum-fuel powered descent and pinpoint soft landing of a reusable launch vehicle. Homework 1 tackled an ascent problem with the indirect approach (Pontryagin Maximum Principle plus shooting); here I use a **direct transcription** instead. The optimal control problem is approximated by a finite-dimensional Nonlinear Programming (NLP) problem through trapezoidal collocation, which is then solved by Sequential Quadratic Programming (SQP). The optional Task 2 adds a Zero-Order Hold transcription and a Successive Convexification (SCvx) loop whose inner sub-problem is solved both as a generic NLP and as a Second-Order Cone Program.

The powered-descent problem models the final phase of flight of a reusable rocket performing a "divert and soft land" manoeuvre, of the kind flown by the SpaceX Falcon 9 first stage and by Mars precision-landing concepts. The vehicle must reach a designated landing pad with zero residual velocity, on minimum propellant, while respecting thrust bounds, a glide-slope corridor, and an altitude constraint.

Why a direct method. I picked a direct transcription over the indirect route of Homework 1 for a few reasons. Indirect methods optimize then discretize: they require the analytical costate equations, the transversality conditions, and a guess for the non-physical initial costates. A direct method discretizes then optimizes, transcribing the trajectory into an NLP that a general-purpose solver handles without any of that machinery. Three things make this convenient here. Path constraints (glide-slope, thrust bounds) enter natively as algebraic inequalities. The converged NLP active set shows the constrained-arc structure of the optimum directly (here the saturated burns of the max-coast-max law). And the convergence radius is much wider, so a crude initial guess is enough. Betts shows that the direct route can even solve high-index differential-algebraic problems on which a hand-reduced indirect formulation fails [1, Sec. 4.12.7]. The price is a larger problem, and the discrete optimum need not satisfy the continuous necessary conditions exactly; the convex reformulations in Task 2 are the route to the real-time guidance that a dense NLP cannot provide [2].

1.1 Vehicle Model and Equations of Motion

The vehicle is modelled as a 2D point mass moving above flat ground with no aerodynamics. The state vector is

$$\mathbf{x}(t) = [x, y, v_x, v_y, m]^T, \quad (1)$$

and the control is the thrust vector $\mathbf{T}(t) = [T_x, T_y]^T$. The equations of motion are

$$\dot{x} = v_x, \quad (2)$$

$$\dot{y} = v_y, \quad (3)$$

$$\dot{v}_x = \frac{T_x}{m}, \quad (4)$$

$$\dot{v}_y = \frac{T_y}{m} - g, \quad (5)$$

$$\dot{m} = -\frac{\|\mathbf{T}\|}{c}, \quad (6)$$

where $g = 9.81 \text{ m/s}^2$ is the gravitational acceleration and $c = I_{sp} g_0$ is the effective exhaust velocity ($I_{sp} = 225 \text{ s}$, $g_0 = 9.80665 \text{ m/s}^2$, hence $c \approx 2206 \text{ m/s}$). As in Homework 1, the problem is non-dimensionalised before being passed to the solver (see Section 3.1); the dimensional form is retained throughout this section for clarity. Compared to Homework 1, the control is the full thrust vector rather than a scalar direction angle, since the magnitude is now also a degree of freedom subject to a bound.

1.2 Boundary Conditions and Mission Parameters

The vehicle starts at a known state and must reach the origin with zero velocity at the prescribed terminal time:

$$\mathbf{x}(0) = [x_0, y_0, v_{x,0}, v_{y,0}, m_0]^T, \quad \mathbf{x}(t_f) = [0, 0, 0, 0, m_f]^T, \quad (7)$$

with the final mass m_f free. The mission data taken from Table 1 of the assignment are summarised in Table 1.

Table 1: Mission data.

Quantity	Symbol	Value	Units
Initial position	(x_0, y_0)	(1000, 3000)	m
Initial velocity	$(v_{x,0}, v_{y,0})$	(300, -200)	m/s
Initial mass	m_0	2000	kg
Specific impulse	I_{sp}	225	s
Thrust bounds	$T_{\min}, T_{\max}$	0, 70	kN
Glide-slope half-angle	$\theta_{\max}$	60	deg
Flight time (fixed)	t_f	38	s

1.3 Path Constraints

Three path constraints are enforced over $[0, t_f]$:

$$T_{\min} \leq \|\mathbf{T}(t)\| \leq T_{\max} \quad (\text{thrust magnitude}), \quad (8)$$

$$|x(t)| \leq \tan(\theta_{\max}) y(t) \quad (\text{glide-slope}), \quad (9)$$

$$y(t) \geq 0 \quad (\text{no underground flight}). \quad (10)$$

The glide-slope condition (9) confines the vehicle to a cone of half-angle $\theta_{\max}$ apexed at the landing pad, ensuring a safe approach corridor. It is convex in (x, y) and decomposes into the linear pair

$$+x - \tan(\theta_{\max}) y \leq 0, \quad -x - \tan(\theta_{\max}) y \leq 0. \quad (11)$$

Both forms are exploited downstream: the linearised pair (11) is what is fed to `fmincon` as a smooth inequality.

1.4 Optimal Control Problem

The minimum-fuel powered-descent problem is the Mayer-form OCP

$$\begin{aligned} \min_{\mathbf{T}(\cdot)} \quad & J = -m(t_f) \\ \text{s.t.} \quad & \text{Eqs. (2)–(6),} \\ & \text{boundary conditions of Section 1.2,} \\ & \text{path constraints (8)–(10).} \end{aligned} \quad (12)$$

The objective is written in **Mayer** form, as a pure terminal cost $J = -m(t_f)$. It could equally be cast in **Lagrange** form through the mass dynamics, $\int_0^{t_f} \|\mathbf{T}\|/c \, dt = m_0 - m(t_f)$; the two are special cases of the **Bolza** objective, and a Lagrange integral cost can always be converted to Mayer form by appending one extra state [1, Sec. 4.4]. I use the Mayer form because the terminal mass is already a decision variable, so it adds nothing to the transcription, whereas the Lagrange form would carry one extra state at every node.

This continuous-time problem is approximated by an NLP using trapezoidal direct collocation, as described in Section 2.

2 Trapezoidal Direct Collocation

Direct transcription methods discretise the continuous optimal control problem (12) over a finite mesh of nodes and replace the continuous-time dynamics with algebraic relations among the node values [3, 1]. The resulting finite-dimensional NLP can be solved by general-purpose nonlinear programming solvers, so I never have to derive the costate equations and transversality conditions of the indirect approach. It is the direct counterpart to the PMP-based formulation used in Homework 1, the same indirect-versus-direct trade-off covered in the course [4, Lecture 8].

2.1 Trapezoidal Quadrature Rule

The continuous-time dynamics are written compactly as

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x} \in \mathbb{R}^{n_x}, \mathbf{u} \in \mathbb{R}^{n_u}. \quad (13)$$

Discretise the time horizon $[0, t_f]$ into N uniformly spaced nodes $\{t_k\}_{k=1}^N$ with spacing $\Delta t = t_f/(N - 1)$, and let

$$\mathbf{x}_k = \mathbf{x}(t_k), \quad \mathbf{u}_k = \mathbf{u}(t_k), \quad \mathbf{f}_k = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k). \quad (14)$$

Integrating the state equation over $[t_k, t_{k+1}]$,

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \int_{t_k}^{t_{k+1}} \mathbf{f}(\mathbf{x}(\tau), \mathbf{u}(\tau)) \, d\tau. \quad (15)$$

Approximating the integrand by a linear interpolation between the two endpoints and applying the **trapezoidal quadrature rule** yields

$$\mathbf{x}_{k+1} - \mathbf{x}_k \approx \frac{\Delta t}{2} (\mathbf{f}_k + \mathbf{f}_{k+1}). \quad (16)$$

The order of the rule follows from a Taylor expansion about t_k : with $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \dot{\mathbf{x}}_k + \frac{1}{2} \Delta t^2 \ddot{\mathbf{x}}_k + O(\Delta t^3)$ and $\frac{\Delta t}{2} (\mathbf{f}_k + \mathbf{f}_{k+1}) = \Delta t \dot{\mathbf{x}}_k + \frac{1}{2} \Delta t^2 \ddot{\mathbf{x}}_k + O(\Delta t^3)$, their difference—the local truncation error—is $\tau_k = -\frac{1}{12} \Delta t^3 \dddot{\mathbf{x}}_k + O(\Delta t^4) = O(\Delta t^3)$; summed over the $O(1/\Delta t)$ intervals this yields a globally second-order, $O(\Delta t^2)$, method [1, Eq. (4.56) and p. 155]. The rule is implicit ($\mathbf{f}_{k+1}$ depends on $\mathbf{x}_{k+1}$) and A-stable: applied to the scalar test equation $\dot{x} = \lambda x$ it gives the amplification factor $x_{k+1}/x_k = (1 + \lambda \Delta t/2)/(1 - \lambda \Delta t/2)$, whose modulus stays below unity for every $\text{Re}(\lambda) < 0$ (the Crank–Nicolson scheme), so stiff modes impose no step-size restriction.

2.2 Defect Constraints

Promoting (16) from approximation to equality constraint produces the **defect equations** of trapezoidal direct collocation:

$$\boldsymbol{\delta}_k := \mathbf{x}_{k+1} - \mathbf{x}_k - \frac{\Delta t}{2} (\mathbf{f}_k + \mathbf{f}_{k+1}) = \mathbf{0}, \quad k = 1, \dots, N - 1. \quad (17)$$

With n_x states this generates $n_x(N - 1)$ scalar nonlinear equality constraints. When all defects are satisfied to machine precision, the discrete trajectory matches the continuous-time dynamics to second order in Δt , and the controls are read as the nodal values of a piecewise-linear input.

For the powered-descent problem ($n_x = 5$, $n_u = 2$) each node contributes $n_x + n_u = 7$ decision variables and the defect block carries $5(N - 1)$ rows.

Higher-order option. The trapezoidal rule is the lowest-order member of the collocation family. Replacing it with the Hermite–Simpson defect

$$\boldsymbol{\delta}_k = \mathbf{x}_{k+1} - \mathbf{x}_k - \frac{\Delta t}{6}(\mathbf{f}_k + 4\bar{\mathbf{f}}_k + \mathbf{f}_{k+1}), \quad \bar{\mathbf{x}}_k = \frac{1}{2}(\mathbf{x}_k + \mathbf{x}_{k+1}) + \frac{\Delta t}{8}(\mathbf{f}_k - \mathbf{f}_{k+1}), \quad (18)$$

with $\bar{\mathbf{f}}_k = \mathbf{f}(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$ evaluated at the interval midpoint, raises the global order from $O(\Delta t^2)$ to $O(\Delta t^4)$ —a three-stage implicit Runge–Kutta (Lobatto IIIA) scheme [1, Eqs. (4.58)–(4.60), p. 147]. The cost is that the midpoint controls $\bar{\mathbf{u}}_k$ become extra NLP unknowns and the path constraints have to be enforced at midpoints as well as nodes. Such a scheme would close the $O(\Delta t^2)$ -versus- $O(\Delta t^4)$ accuracy gap observed against the RK4 transcription in Table 7.

2.3 NLP Formulation

The decision vector stacks states and controls node by node:

$$\mathbf{z} = [\mathbf{x}_1^T, \mathbf{u}_1^T, \mathbf{x}_2^T, \mathbf{u}_2^T, \dots, \mathbf{x}_N^T, \mathbf{u}_N^T]^T \in \mathbb{R}^{(n_x+n_u)N}. \quad (19)$$

The transcribed problem is

$$\begin{aligned} \min_{\mathbf{z}} \quad & -m_N \\ \text{s.t.} \quad & \mathbf{c}_{\text{eq}}^{\text{lin}}(\mathbf{z}) = \mathbf{0} \quad (\text{boundary conditions}), \\ & \mathbf{c}_{\text{eq}}^{\text{nl}}(\mathbf{z}) = \mathbf{0} \quad (\text{defects}), \\ & \mathbf{c}_{\text{ineq}}(\mathbf{z}) \leq \mathbf{0} \quad (\text{path constraints}), \\ & \mathbf{z}_{\text{lb}} \leq \mathbf{z} \leq \mathbf{z}_{\text{ub}} \quad (\text{box bounds}). \end{aligned} \quad (20)$$

With $N = 50$, the dimensions are: $7N = 350$ decision variables; 9 linear equalities (5 initial + 4 final boundary conditions, m_f being free); $5(N - 1) = 245$ nonlinear equalities (defects); $4N = 200$ nonlinear inequalities (lower and upper thrust magnitude, two glide-slope half-spaces); and $7N$ box bounds.

2.4 KKT Optimality Conditions

Introducing multipliers $\boldsymbol{\lambda}^{\text{eq}}$ and $\boldsymbol{\mu} \geq \mathbf{0}$ for the equality and inequality constraints, the Lagrangian of (20) is

$$\mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}^{\text{eq}}, \boldsymbol{\mu}) = J(\mathbf{z}) + (\boldsymbol{\lambda}^{\text{eq}})^T \mathbf{c}_{\text{eq}}(\mathbf{z}) + \boldsymbol{\mu}^T \mathbf{c}_{\text{ineq}}(\mathbf{z}). \quad (21)$$

Under a constraint qualification such as the linear independence of the active constraint gradients (LICQ), a necessary condition for $\mathbf{z}^*$ to be a local minimiser is the existence of multipliers $(\boldsymbol{\lambda}^{\text{eq},*}, \boldsymbol{\mu}^*)$ satisfying the **Karush–Kuhn–Tucker** (KKT) conditions [5, Ch. 12]:

$$\nabla_{\mathbf{z}} \mathcal{L}(\mathbf{z}^*, \boldsymbol{\lambda}^{\text{eq},*}, \boldsymbol{\mu}^*) = \mathbf{0} \quad (\text{stationarity}), \quad (22)$$

$$\mathbf{c}_{\text{eq}}(\mathbf{z}^*) = \mathbf{0}, \quad \mathbf{c}_{\text{ineq}}(\mathbf{z}^*) \leq \mathbf{0} \quad (\text{primal feasibility}), \quad (23)$$

$$\boldsymbol{\mu}^* \geq \mathbf{0} \quad (\text{dual feasibility}), \quad (24)$$

$$\mu_j^* c_{\text{ineq},j}(\mathbf{z}^*) = 0, \quad \forall j \quad (\text{complementary slackness}). \quad (25)$$

Complementarity says that an inequality is either active ($c_j = 0$, with $\mu_j > 0$) or strictly inactive ($c_j < 0$, with $\mu_j = 0$). At the optimum of the powered-descent problem the upper thrust bound $\|\mathbf{T}\| = T_{\text{max}}$ is active on the two burn arcs (about half of the nodes), which matches the max–coast–max structure of the fuel-optimal control law for linear-in-control dynamics. The corresponding KKT multipliers are positive, so they flag the binding constraint that shapes the solution.

2.5 Fuel-Optimal Thrust Structure

Even though the problem is solved directly, its optimal thrust profile carries the structure that the maximum principle predicts for dynamics linear in the control. Forming the Hamiltonian of the continuous OCP (12),

$$\mathcal{H} = \lambda_x v_x + \lambda_y v_y + \frac{\lambda_{v_x} T_x + \lambda_{v_y} T_y}{m} - \lambda_{v_y} g - \lambda_m \frac{\|\mathbf{T}\|}{c}, \quad (26)$$

and aligning the thrust direction with the primer vector $\mathbf{p} = (\lambda_{v_x}, \lambda_{v_y})/m$, the Hamiltonian becomes linear in the thrust magnitude $\|\mathbf{T}\| \in [T_{\text{min}}, T_{\text{max}}]$, with coefficient the switching function

$$S = \|\mathbf{p}\| - \frac{\lambda_m}{c}. \quad (27)$$

The optimal magnitude therefore lies at a bound— T_{max} or T_{min} according to the sign of S , with a singular arc only if $S \equiv 0$ on an interval—producing the **max–coast–max** (bang–off–bang) profile [6, Sec. 3.9]. This is the same bang–bang mechanism behind the bilinear-tangent law of Homework 1; here it shows up numerically as the active set of the upper thrust bound (Section 2.4) and is visible in the thrust histories of Sections 4 and 6.

2.6 Discrete Multipliers as Costate Estimates

The NLP multipliers are not just algebraic by-products: as the mesh is refined they converge to scaled values of the continuous costates, the covector mapping property.

For the trapezoidal transcription, the discrete multipliers $\boldsymbol{\alpha}_k$ of the path constraints map to the continuous path-constraint adjoint $\hat{\boldsymbol{\mu}}_k$ (distinct from the inequality multipliers $\boldsymbol{\mu}$ of Section 2.4) through [1, Sec. 4.11.2, Eqs. (4.235)–(4.237)]

$$\hat{\boldsymbol{\mu}}_1 = -\frac{2}{h_1} \boldsymbol{\alpha}_1, \quad \hat{\boldsymbol{\mu}}_k = -\frac{2}{h_{k-1} + h_k} \boldsymbol{\alpha}_k, \quad \hat{\boldsymbol{\mu}}_M = -\frac{2}{h_{M-1}} \boldsymbol{\alpha}_M, \quad (28)$$

with h_k the local step. This is why the converged active set is physically meaningful: the nodes where the thrust-bound multiplier is non-zero are the powered (constrained) arcs of the continuous solution, so the NLP recovers the max–coast–max structure of Section 2.5 without ever forming the costates explicitly.

2.7 Sequential Quadratic Programming

The NLP (20) is solved with MATLAB’s `fmincon` configured with the SQP algorithm [5, Ch. 18]. SQP iteratively constructs a quadratic-programming approximation of the KKT system: at each iterate $\mathbf{z}^{(k)}$ the search direction $\mathbf{d}^{(k)}$ is found by solving

$$\begin{aligned} \min_{\mathbf{d}} \quad & \frac{1}{2} \mathbf{d}^T \mathbf{H}^{(k)} \mathbf{d} + \nabla J(\mathbf{z}^{(k)})^T \mathbf{d} \\ \text{s.t.} \quad & \nabla \mathbf{c}_{\text{eq}}(\mathbf{z}^{(k)})^T \mathbf{d} + \mathbf{c}_{\text{eq}}(\mathbf{z}^{(k)}) = \mathbf{0}, \\ & \nabla \mathbf{c}_{\text{ineq}}(\mathbf{z}^{(k)})^T \mathbf{d} + \mathbf{c}_{\text{ineq}}(\mathbf{z}^{(k)}) \leq \mathbf{0}, \end{aligned} \quad (29)$$

where $\mathbf{H}^{(k)}$ is a quasi-Newton (BFGS) approximation of the Hessian of the Lagrangian. The next iterate is $\mathbf{z}^{(k+1)} = \mathbf{z}^{(k)} + \alpha^{(k)} \mathbf{d}^{(k)}$, with the step length $\alpha^{(k)}$ obtained from a line search on a merit function that balances objective decrease against constraint violation.

The Jacobian of the defect block (17) is highly sparse: each defect $\boldsymbol{\delta}_k$ couples only the two adjacent nodes k and $k+1$, through constant $\pm \mathbf{I}$ blocks plus the nodal values of $\mathbf{f}$, giving a block-bidiagonal structure. Betts uses this sparsity, grouping the finite-difference perturbations so that the cost of building the Jacobian scales with the bandwidth rather than the full variable count [1, Sec. 4.6]. Here `fmincon` estimates a dense finite-difference Jacobian, which is fine at $N = 50$ (350 variables) but is the first thing I would replace—by supplying the sparsity pattern—to scale the mesh up.

3 Numerical Implementation

This section details the practical setup of the NLP (20) in MATLAB. The full transcription is built and solved by the script `main_task1.m`; `fmincon` is the only solver dependency (no CasADi, no YALMIP). Figure 1 shows how the script turns the OCP into an NLP and solves it.

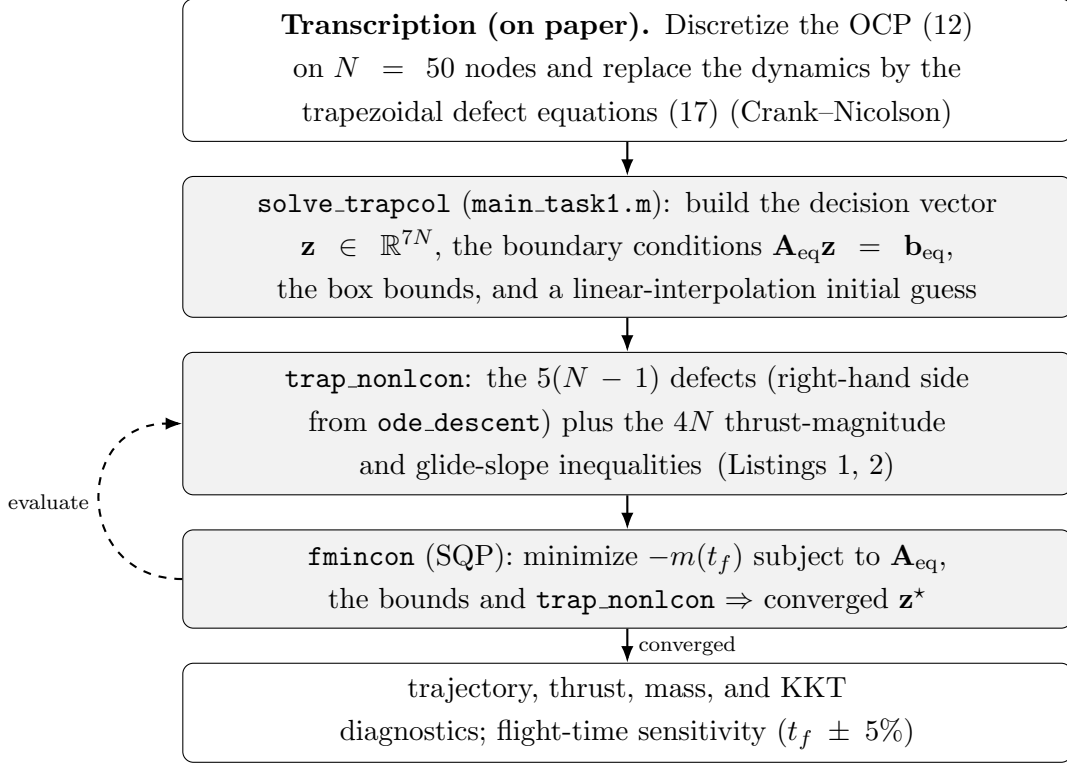


Figure 1: Task 1 solution pipeline: trapezoidal direct collocation turns the OCP into an NLP, which `fmincon` solves by SQP. Listing numbers refer to Appendix A.

3.1 Non-Dimensionalisation

The decision-vector entries span four orders of magnitude in SI units (positions $\sim 10^3$ m, velocities $\sim 10^2$ m/s, mass $\sim 10^3$ kg, thrust components $\sim 7 \cdot 10^4$ N). Mixing scales of this size in a single NLP gives a badly conditioned Jacobian and slow convergence under finite-difference gradients. As in Homework 1, I non-dimensionalise the problem before handing it to `fmincon`. The reference scales are listed in Table 2.

Table 2: Non-dimensionalisation reference scales.

Quantity	Symbol	Definition	Numerical value
Length	L_{ref}	y_0	3000 m
Acceleration	a_{ref}	g	9.81 m/s ²
Time	t_{ref}	$\sqrt{L_{ref}/g}$	17.49 s
Velocity	V_{ref}	$\sqrt{g L_{ref}}$	171.6 m/s
Mass	m_{ref}	m_0	2000 kg
Thrust	T_{ref}	$m_0 g$	19 620 N

In non-dimensional variables (denoted by tildes), the dynamics (2)–(6) reduce to

$$\dot{\tilde{x}} = \tilde{v}_x, \quad \dot{\tilde{y}} = \tilde{v}_y, \quad (30)$$

$$\dot{\tilde{v}}_x = \frac{\tilde{T}_x}{\tilde{m}}, \quad \dot{\tilde{v}}_y = \frac{\tilde{T}_y}{\tilde{m}} - 1, \quad (31)$$

$$\dot{\tilde{m}} = -V_c \|\tilde{\mathbf{T}}\|, \quad (32)$$

where $V_c = V_{ref}/c \approx 0.0778$ is the only residual dimensionless parameter—the non-dimensional mass-flow coefficient ($\dot{\tilde{m}} = -V_c \|\tilde{\mathbf{T}}\|$), i.e. the ratio of the characteristic flight speed to the effective exhaust velocity. It is not the Tsiolkovsky number itself: the rocket-equation ratio $\Delta v/c = \ln(m_0/m_f)$ only appears once $V_c \|\tilde{\mathbf{T}}\|$ is integrated over a burn [2, Sec. 3.8.3]. All decision variables now span $O(1)$ magnitudes, with positions in $[-1/3, 1]$, velocities in $[-1.17, 1.75]$, mass in $[10^{-3}, 1]$, and thrust components in $[-3.57, 3.57]$; the Jacobians of the LTV linearisation (Section 6) become uniformly $O(1)$, which improves the conditioning of `fmincon`’s SQP iterations and of the SCvx outer loop. Boundary conditions, glide-slope angle, and the time horizon t_f are all rescaled by their natural units; results are returned to SI before printing and plotting.

3.2 Decision Vector Layout

With $N = 50$ collocation nodes, the decision vector (19) has $7N = 350$ components ordered as

$$\mathbf{z} = \left[\underbrace{x_1, y_1, v_{x,1}, v_{y,1}, m_1, T_{x,1}, T_{y,1}}_{\text{node 1}}, \dots, \underbrace{x_N, y_N, v_{x,N}, v_{y,N}, m_N, T_{x,N}, T_{y,N}}_{\text{node } N} \right]^T. \quad (33)$$

A slice operator `idx(i) = (i-1)*7 + (1:7)` returns the indices of the seven entries belonging to node i , used throughout the helper functions to build sparse constraint matrices and unpack the converged solution.

3.3 Initial Guess

A reasonable initial guess matters a lot for SQP convergence on this nonconvex NLP. The state variables are linearly interpolated between the boundary states with $\alpha = (i - 1)/(N - 1)$:

$$x_i^{(0)} = (1 - \alpha) x_0, \quad y_i^{(0)} = (1 - \alpha) y_0, \quad (34)$$

$$v_{x,i}^{(0)} = (1 - \alpha) v_{x,0}, \quad v_{y,i}^{(0)} = (1 - \alpha) v_{y,0}, \quad (35)$$

$$m_i^{(0)} = m_0(1 - 0.3\alpha). \quad (36)$$

The control guess is a constant hover thrust $\mathbf{T}^{(0)} = (0, m_0 g)^T$, which approximately balances gravity at the start and provides a search direction that does not violate the thrust-magnitude bounds.

3.4 Constraints

Linear equality (boundary conditions). The nine scalar conditions are encoded in $\mathbf{A}_{\text{eq}}\mathbf{z} = \mathbf{b}_{\text{eq}}$ as a sparse $9 \times 7N$ matrix: five rows fix the initial state and four rows fix the terminal $(x, y, v_x, v_y) = (0, 0, 0, 0)$. The terminal mass is left free; its strictly-positive lower bound is enforced as a box constraint (see below).

Nonlinear equality (defects). The function `trap_nonlcon` reshapes $\mathbf{z}$ into a $7 \times N$ matrix, evaluates the right-hand side $\mathbf{f}_k$ at every node via `dyn_rhs`, and assembles the $5(N - 1) = 245$ defect entries of Eq. (17).

Nonlinear inequality. The same routine returns the $4N$ inequalities:

- Thrust magnitude (lower and upper): $T_{\min} - \|\mathbf{T}_k\| \leq 0$ and $\|\mathbf{T}_k\| - T_{\max} \leq 0$, both nonsmooth at the origin but smoothly differentiable away from $\mathbf{T} = \mathbf{0}$;
- Glide-slope (linear, decomposed): $+x_k - \tan \theta_{\max} y_k \leq 0$ and $-x_k - \tan \theta_{\max} y_k \leq 0$.

Box bounds. Per-node bounds enforce $\tilde{y}_k \geq 0$, $10^{-3} \leq \tilde{m}_k \leq 1$ (equivalently $2 \leq m_k \leq m_0$ kg in SI), and $|\tilde{T}_{x,k}|, |\tilde{T}_{y,k}| \leq \tilde{T}_{\max}$. The strictly-positive mass lower bound avoids division-by-zero in the dynamics RHS (4)–(5); the chosen value of 2 kg is well below the propellant inventory at any feasible terminal state.

3.5 Solver Settings

The NLP is solved with `fmincon` using the options listed in Table 3.

Table 3: `fmincon` options.

Option	Value
Algorithm	'sqp'
MaxIterations	1000
MaxFunctionEvaluations	10^6
OptimalityTolerance	10^{-5}
ConstraintTolerance	10^{-6}
StepTolerance	10^{-10}

I chose the SQP algorithm over the interior-point method (`fmincon`'s default) because it tracks the active-set structure of the max-coast-max optimum — the thrust bound binds on entire arcs — better than barrier-based interior-point iterations, and it handles the equality-dense defect block well. Gradients are not provided explicitly, so `fmincon` computes them via finite differences; the typical wall time at $N = 50$ is about one minute per solve on the test laptop.

One note on the convergence tail: on the coast arc the optimal control sits exactly at $\mathbf{T} = \mathbf{0}$, which is the nonsmooth point of $\|\mathbf{T}\|$ in the mass equation (6) and in the thrust-magnitude constraint. Under finite-difference gradients this caps the attainable first-order optimality at the 10^{-3} – 10^{-4} level (non-dim) and explains why some runs stop at the iteration limit with feasible, good-enough iterates rather than at the optimality tolerance (Sections 4–5). Two fixes I left to the roadmap are analytical gradients with a smoothed norm, or the lossless-convexification slack $\Gamma \geq \|\mathbf{T}\|$ of Açikmeşe–Ploen [7], which removes the norm from the dynamics altogether.

4 Numerical Results

The NLP (20) is solved at the nominal flight time $t_f = 38$ s with $N = 50$ collocation nodes. `fmincon` runs for roughly a minute of wall time on the test laptop and returns a final mass of $m_f = 1403.20$ kg, equivalently 596.80 kg of propellant consumed (about 30% of the initial mass). The solver terminates at the `MaxIterations` cap of 1000 iterations; the constraint residual is below the tolerance of 10^{-6} and the first-order optimality plateaus at $\sim 10^{-3}$ – 10^{-4} in non-dim units, so the algorithm has found a good local optimum but has not fully converged. The plateau is consistent with the nonsmooth coast arc discussed in Section 3 (see Section 5 for the same behaviour across the sensitivity sweep).

The four key plots are presented below. All three sensitivity runs (Section 5) are overlaid on the same axes for reference; the discussion in this section focuses on the nominal $t_f = 38.00$ s curve.

4.1 Trajectory and Glide-Slope Corridor

Figure 2 shows the descent trajectory in the (x, y) plane together with the glide-slope corridor (dashed grey lines bounding the cone of half-angle $\theta_{\max} = 60^\circ$). The vehicle enters the corridor from the upper right with significant horizontal velocity, executes a divert manoeuvre to align with the landing pad, and softly touches down at the origin (black triangle). The trajectory approaches the corridor boundary near the apex of the divert turn, at about one third of the flight, but does not reach it: the glide-slope constraint stays inactive over the whole descent, with a small margin, as quantified in Section 4.3.

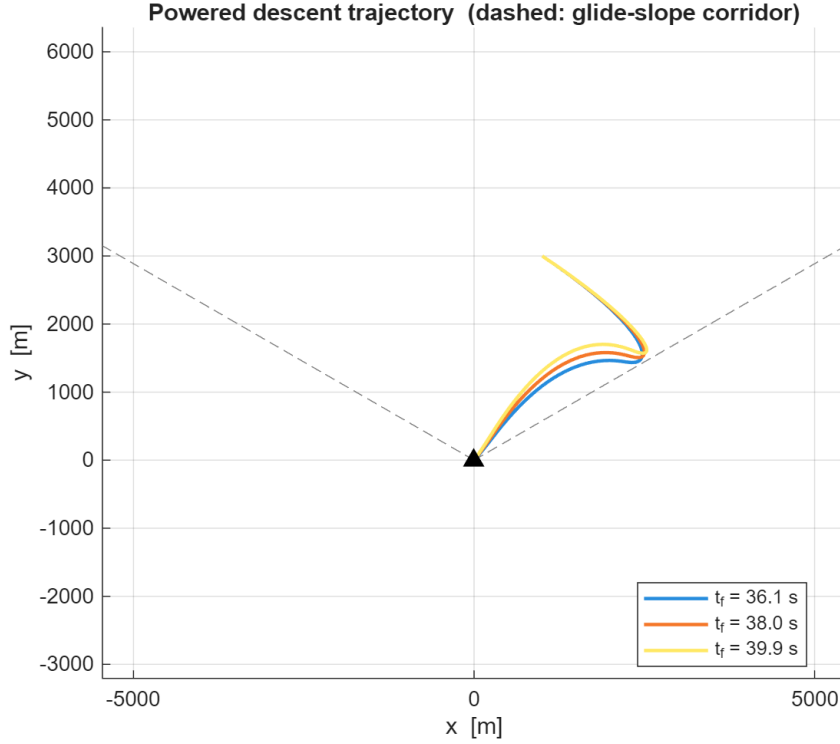


Figure 2: Powered descent trajectory in the (x, y) plane. Dashed grey lines: glide-slope corridor at $\theta_{\max} = 60^\circ$. Black triangle: landing pad at the origin.

4.2 Thrust Magnitude

Figure 3 shows the thrust-magnitude profile $\|\mathbf{T}(t)\|$. The fuel-optimal solution shows the three-arc **max-coast-max** (bang-off-bang) structure: an initial braking burn at $\|\mathbf{T}\| = T_{\max} = 70$ kN cancels most of the divert velocity, the thrust then drops to zero along a midcourse coast arc, and a second saturated burn does the terminal braking down to the soft touchdown. For the nominal $t_f = 38$ s the switches occur at $t \approx 14.0$ s and $t \approx 33.1$ s, leaving a coast arc of about 19 s. This is the discrete-time version of the same Pontryagin optimality principle that produced the bilinear-tangent law in Homework 1, applied here to a problem with linear-in-control dynamics and a control-magnitude bound (the switching-function argument of Section 2.5): the optimal thrust magnitude never sits at intermediate values, but switches between the extremes of its admissible set ($T_{\max}$ and the lower bound $T_{\min} = 0$).

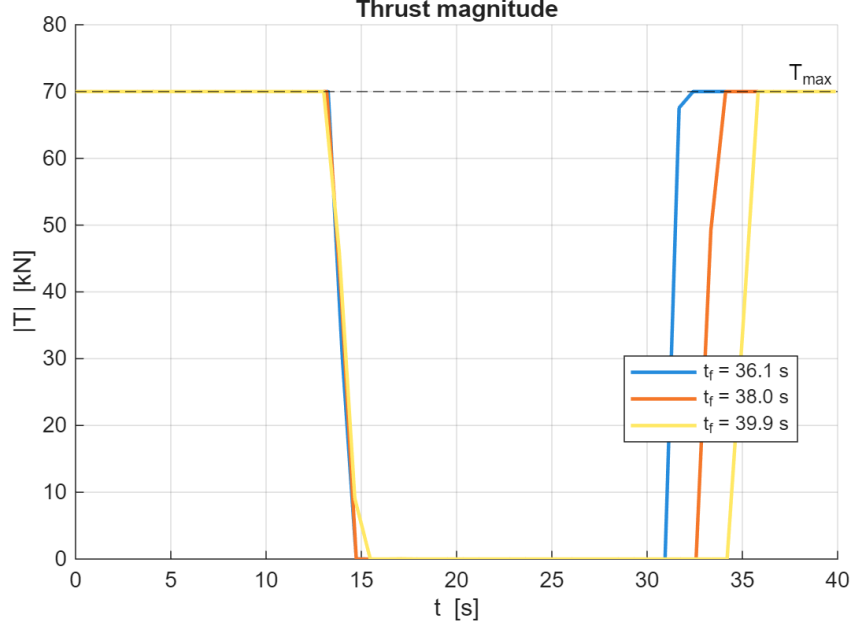
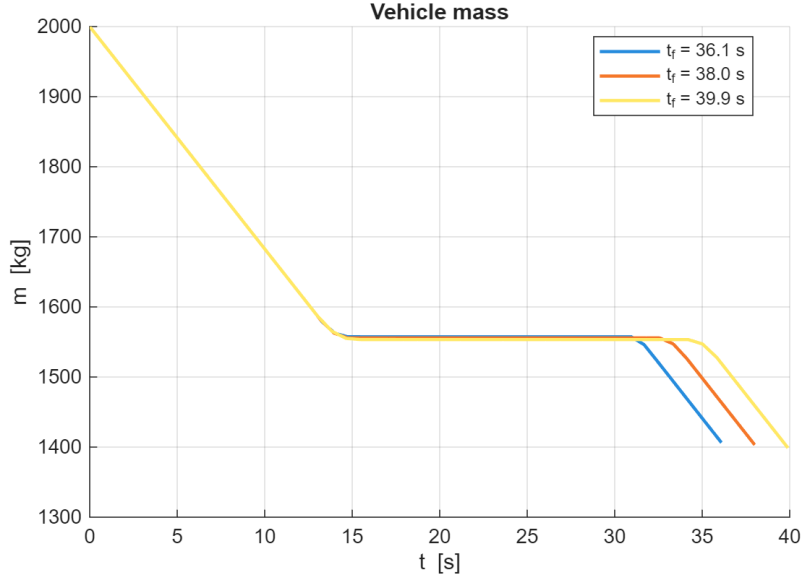


Figure 3: Thrust-magnitude profile. Dashed line: upper bound $T_{\max} = 70$ kN.

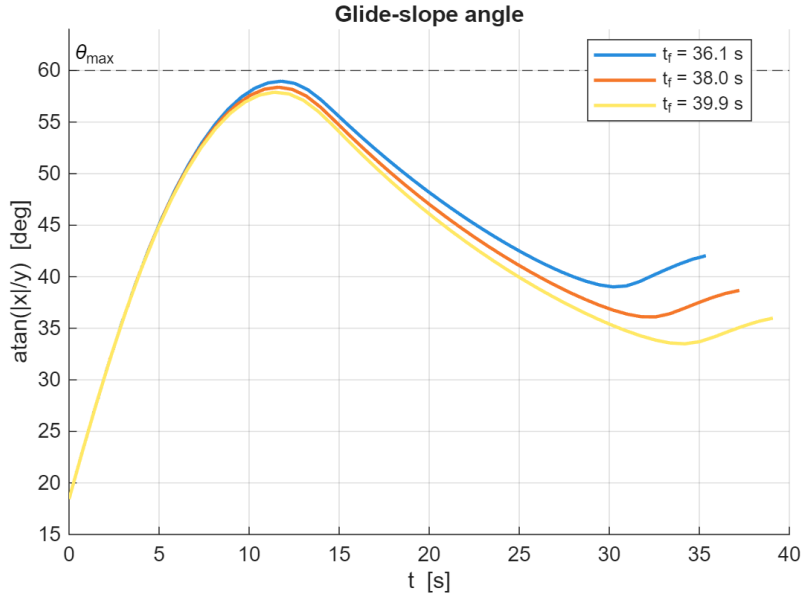
4.3 Mass and Glide-Slope Angle

Figure 4a shows the vehicle mass $m(t)$, which decreases at a rate proportional to $\|\mathbf{T}\|$ (Eq. 6): the slope is steep and approximately constant during the two saturated burns and the mass stays constant along the midcourse coast arc, mirroring the max-coast-max thrust profile.

Figure 4b shows the instantaneous glide-slope angle $\theta(t) = \arctan(|x|/y)$, bounded above by $\theta_{\max} = 60^\circ$ (dashed). The angle peaks at $\theta \approx 58.4^\circ$ around $t \approx 12$ s, near the apex of the divert turn, leaving a minimum margin of $\approx 1.6^\circ$: the glide-slope constraint is never active at the optimum. The KKT multipliers returned by `fmincon` confirm this: the multipliers of the two glide-slope half-space rows are numerically zero at every node, while the upper thrust-magnitude bound carries strictly positive multipliers on the two burn arcs — by complementary slackness, the thrust bound is the only path constraint shaping the solution. Nodes below 1 m altitude are omitted from the plot: as $(x, y) \rightarrow (0, 0)$ the ratio $|x|/y$ is numerically indeterminate, and millimetre-level residuals (well within the `ConstraintTolerance` of the linear form (11)) would produce arbitrary spurious angles.



(a) Vehicle mass.



(b) Glide-slope angle $\arctan(|x|/y)$ (nodes below 1 m altitude omitted).

Figure 4: Mass profile and instantaneous glide-slope angle for the nominal solution.

4.4 Grid Convergence

To verify that $N = 50$ nodes are adequate, the nominal problem is re-solved on coarser and finer grids. For each N the converged piecewise-linear control is replayed through the continuous-time dynamics with `ode45` (the same fidelity metric used in Section 6.5) and the maximum position+velocity node error is recorded. The results are summarised in Table 4.

Table 4: Grid-convergence study at the nominal $t_f = 38$ s.

N	m_f [kg]	Max node error [-]	Wall time [s]
25	1402.47	$6.39 \cdot 10^{-3}$	8
50	1403.20	$1.55 \cdot 10^{-3}$	65
100	1402.42	$3.81 \cdot 10^{-4}$	90

The replay error drops by about a factor of four per mesh halving, which matches the $O(\Delta t^2)$ global order of the trapezoidal rule. The final mass fluctuates within ± 0.8 kg ($\approx 0.13\%$ of the propellant) with no systematic trend: the cost landscape is flat enough that the three grids stop at slightly different iterates of comparable quality (the $N = 25$ and $N = 100$ runs stop on the step-size criterion, the $N = 50$ run at the iteration cap). $N = 50$ is retained as the working compromise, with the forward-integration assessment of Section 6.5 as the fidelity cross-check.

5 Sensitivity Analysis

To see how fuel consumption depends on the imposed flight time, I re-solve the NLP at $t_f \in \{0.95, 1.00, 1.05\} \cdot 38$ s, i.e., at 36.10, 38.00, and 39.90 s. Each solve uses the linear-interpolation initial guess of Section 3.3 (no warm-start across runs). The shortest and nominal runs ($t_f = 36.10$ s and 38.00 s) stop at the **MaxIterations** cap with first-order optimality of order 10^{-3} – 10^{-4} in the non-dim cost; the longest run ($t_f = 39.90$ s) meets the step-size convergence criterion. The two capped runs still satisfy all constraints to within the **ConstraintTolerance** of 10^{-6} , so I report them as engineering-quality solutions rather than fully optimised ones.

Table 5 reports the converged final masses and propellant consumption.

Table 5: Sensitivity sweep on flight time.

t_f [s]	m_f [kg]	Fuel consumed [kg]
36.10	1406.33	593.67
38.00	1403.20	596.80
39.90	1398.82	601.18

Fuel use grows with t_f across the swept window. The reason is simple: a longer descent keeps the vehicle in the gravity field longer, so more thrust goes into counteracting gravity (gravity loss). Within the $\pm 5\%$ window around the nominal 38 s the variation is small (≈ 7.5 kg, or about 1.3% of the propellant), so the nominal t_f sits near a flat region of the cost—which fits the fact that the mission is dominated by horizontal divert and not by hover time.

The change in flight time is absorbed almost entirely by the coast arc. Table 6 shows this through the switching times, computed as the crossings of the thrust magnitude through $0.5 T_{\max}$: the first burn barely moves across the sweep, while the start of the terminal burn shifts to track the imposed t_f .

Table 6: Thrust switching structure across the sensitivity sweep.

t_f [s]	Burn 1 ends [s]	Burn 2 starts [s]	Coast length [s]
36.10	13.91	31.32	17.41
38.00	14.04	33.12	19.09
39.90	14.08	35.04	20.95

The monotonic trend suggests that the fuel-optimal solution sits at the smallest t_f the thrust-magnitude bound allows. A free-time formulation, where t_f becomes a decision variable, would find this minimum on its own; this is one of the extensions listed in Section 7.

The trajectory, thrust, mass, and glide-slope plots in Section 4 overlay the three sensitivity runs on common axes for direct comparison. The solutions look much alike: all three show the max–coast–max thrust structure, all three stay inside the glide-slope corridor, and the trajectory shapes differ only slightly, with the extra flight time spent coasting at altitude before the terminal burn.

6 Task 2: Zero-Order Hold Discretization

The optional Task 2 of the assignment asks for an alternative transcription of the powered-descent problem based on a **Zero-Order Hold** (ZOH) discretisation, validated by forward-integrating the optimised control schedule with a high-fidelity integrator. I present three implementations here and compare them against the trapezoidal-collocation baseline of Task 1:

- (a) **Nonlinear ZOH with RK4 propagation** – a multiple-shooting NLP that holds $\mathbf{u}$ piecewise constant and propagates the nonlinear dynamics between grid nodes via a fixed-step RK4 integrator;
- (b) **LTV-linearised ZOH with SCvx outer loop, solved by fmincon/SQP** – the literal Appendix A construction: linearise about a reference, compute the discrete-time matrices via the auxiliary ODE, solve the resulting LTV optimisation, and iterate;
- (c) **LTV-linearised ZOH with SCvx outer loop, solved by YALMIP+ECOS** – same outer loop as variant (b), but the inner sub-problem is modelled in YALMIP as a Second-Order Cone Program (SOCP) and solved by the conic solver ECOS, exposing the convex structure that variant (b) hides inside a general-purpose NLP.

6.1 ZOH Transcription for LTV Systems (Appendix A)

Appendix A of the assignment derives the ZOH transcription for a continuous-time linear time-varying system

$$\dot{\mathbf{x}}(t) = \mathbf{A}(t) \mathbf{x}(t) + \mathbf{B}(t) \mathbf{u}(t) + \mathbf{c}(t). \quad (37)$$

Discretising time on a uniform grid $\{t_k\}$ with spacing Δt and assuming the control to be piecewise constant on each interval,

$$\mathbf{u}(t) = \mathbf{u}_k \quad \text{for } t \in [t_k, t_{k+1}), \quad (38)$$

the variation-of-constants formula yields the discrete-time recursion

$$\mathbf{x}_{k+1} = \bar{\mathbf{A}}_k \mathbf{x}_k + \bar{\mathbf{B}}_k \mathbf{u}_k + \bar{\mathbf{c}}_k, \quad (39)$$

where the discrete-time matrices follow from the state-transition matrix $\Phi(t, t_k)$:

$$\bar{\mathbf{A}}_k = \Phi(t_{k+1}, t_k), \quad \bar{\mathbf{B}}_k = \int_{t_k}^{t_{k+1}} \Phi(t_{k+1}, s) \mathbf{B}(s) \, ds, \quad \bar{\mathbf{c}}_k = \int_{t_k}^{t_{k+1}} \Phi(t_{k+1}, s) \mathbf{c}(s) \, ds. \quad (40)$$

These are evaluated numerically by integrating an augmented system over each interval $[t_k, t_{k+1}]$. Defining $\hat{\mathbf{B}}(t) = \int_{t_k}^t \Phi(t, s) \mathbf{B}(s) ds$ and $\hat{\mathbf{c}}(t) = \int_{t_k}^t \Phi(t, s) \mathbf{c}(s) ds$, one obtains the linear ODEs

$$\dot{\Phi} = \mathbf{A} \Phi, \quad \dot{\hat{\mathbf{B}}} = \mathbf{A} \hat{\mathbf{B}} + \mathbf{B}, \quad \dot{\hat{\mathbf{c}}} = \mathbf{A} \hat{\mathbf{c}} + \mathbf{c}, \quad (41)$$

with $\Phi(t_k, t_k) = \mathbf{I}$ and $\hat{\mathbf{B}}(t_k) = \mathbf{0}$, $\hat{\mathbf{c}}(t_k) = \mathbf{0}$. The ODE for $\hat{\mathbf{B}}$ comes from differentiating its defining integral with the Leibniz rule; the integrand depends on the upper limit t both explicitly and through $\Phi(t, s)$:

$$\dot{\hat{\mathbf{B}}}(t) = \Phi(t, t) \mathbf{B}(t) + \int_{t_k}^t \frac{\partial \Phi(t, s)}{\partial t} \mathbf{B}(s) ds = \mathbf{B}(t) + \mathbf{A}(t) \hat{\mathbf{B}}(t), \quad (42)$$

using $\Phi(t, t) = \mathbf{I}$ and $\partial_t \Phi = \mathbf{A} \Phi$; the same argument yields the $\hat{\mathbf{c}}$ equation. At the end of each interval $\bar{\mathbf{A}}_k = \Phi(t_{k+1}, t_k)$, $\bar{\mathbf{B}}_k = \hat{\mathbf{B}}(t_{k+1})$, $\bar{\mathbf{c}}_k = \hat{\mathbf{c}}(t_{k+1})$. The semigroup identity $\Phi(t_{k+1}, s) = \Phi(t_{k+1}, t_k) \Phi(t_k, s)$ makes this single forward sweep mathematically equivalent to the β, γ form of the appendix and avoids any need to invert Φ [8]. A piecewise-linear (first-order hold) input is handled by the same machinery with one additional auxiliary integral.

6.2 Application to the Nonlinear Powered-Descent Problem

The dynamics (2)–(6) are nonlinear in $(\mathbf{x}, \mathbf{u})$ because of the T_x/m , T_y/m , and $\|\mathbf{T}\|/c$ terms, so Appendix A does not apply directly. I pursue two strategies:

Variant (a): nonlinear ZOH with exact RK4 propagation

Keep the nonlinear dynamics intact and propagate the state from $\mathbf{x}_k$ to $\mathbf{x}_{k+1}$ using the constant control $\mathbf{u}_k$:

$$\mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k; \Delta t), \quad (43)$$

where $F(\cdot, \cdot; \Delta t)$ is the ZOH-input flow map of the nonlinear ODE, implemented as a fixed-step RK4 with $n_{\text{sub}} = 2$ substeps per interval (`rk4_zoh.m`, Listing 3). The decision vector and constraint structure are identical to Task 1 except that the trapezoidal defects (17) are replaced by the nonlinear ZOH defects:

$$\delta_k^{\text{ZOH}} := \mathbf{x}_{k+1} - F(\mathbf{x}_k, \mathbf{u}_k; \Delta t) = \mathbf{0}, \quad k = 1, \dots, N-1, \quad (44)$$

and the terminal control $\mathbf{u}_N$ is pinned to zero. Under the ZOH (piecewise-constant) convention each $\mathbf{u}_k$ is held over $[t_k, t_{k+1})$, so the N controls drive only $N-1$ intervals and $\mathbf{u}_N$ has no interval left to act on—unlike the trapezoidal transcription, where all N nodal controls define a piecewise-linear input. The decision-vector layout (19) is unchanged; only the interpretation of the last control block differs (a first-order-hold convention would instead restore a meaning to $\mathbf{u}_N$).

Variant (b): linearised LTV ZOH with successive convexification

Linearise the dynamics about a reference trajectory $(\mathbf{x}_{\text{ref}}(t), \mathbf{u}_{\text{ref}}(t))$,

$$\mathbf{A}(t) = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\text{ref}}, \quad \mathbf{B}(t) = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\text{ref}}, \quad \mathbf{c}(t) = \mathbf{f}(\mathbf{x}_{\text{ref}}, \mathbf{u}_{\text{ref}}) - \mathbf{A} \mathbf{x}_{\text{ref}} - \mathbf{B} \mathbf{u}_{\text{ref}}, \quad (45)$$

yielding the LTV form (37). The discrete matrices $\bar{\mathbf{A}}_k, \bar{\mathbf{B}}_k, \bar{\mathbf{c}}_k$ are computed via Eq. (41) on every interval and used as linear equality constraints (39) in the NLP. With linear dynamics, linear box bounds, linear glide-slope, and a convex thrust-magnitude bound, the LTV sub-problem is a Second-Order Cone Program (SOCP), the convex class detailed in Section 6.3.

The linearisation is only locally accurate, so I wrap a **Successive Convexification** (SCvx) outer loop around it [4, Lecture 12]: at each iteration the LTV sub-problem is solved, the reference is updated to the new solution, and the cycle repeats until $\|\mathbf{x}^{(j+1)} - \mathbf{x}^{(j)}\|_2 < \tau$. The canonical SCvx stopping test has three parts: small iterate change, small nonlinear dynamics residual, and vanishing virtual buffers [2, Secs. 3.5–3.7]. The `ode45` replay ratio test introduced below is the dynamics-residual component of that test. Two implementation choices mattered for stable convergence on this problem:

- **Warm start from the trapezoidal solution.** Cold-starting SCvx from a linear-interpolation reference with hover thrust drives the LTV NLP into regions where the linearisation predicts mass creation ($\dot{m} > 0$) and other unphysical behaviour, because the offset $\mathbf{c}(t)$ is built from the reference dynamics and only matches the nonlinear $\mathbf{f}(\mathbf{x}, \mathbf{u})$ near the linearisation point. Warm-starting from the Task 1 trapezoidal solution places the initial reference close to the optimum, so the linearisation holds from the first iteration.
- **Adaptive box trust region with ratio test.** Even with a good warm start, an unconstrained LTV NLP can drift along directions where the linear model is loose (in particular the mass equation, since $\partial f_5 / \partial \mathbf{T}$ is nearly singular when $\|\mathbf{T}\|$ is small): the LTV “predicts” a fuel saving that the nonlinear dynamics do not deliver, and successive iterations oscillate around the true optimum. I therefore wrap the LTV NLP in a hard box trust region with adaptive scale $\rho \in [\rho_{\min}, 1]$ —an adaptive hard trust region in the taxonomy of [2, Secs. 3.5–3.7]—namely $|\tilde{\mathbf{z}} - \tilde{\mathbf{z}}_{\text{ref}}| \leq \rho \boldsymbol{\rho}_0$, with base radii $\boldsymbol{\rho}_0 = (\rho_{xy}, \rho_v, \rho_m, \rho_T) = (0.17, 0.6, 0.1, 1.0)$ in non-dim units (roughly 500 m, 100 m/s, 200 kg, 20 000 N).
 - After each candidate step $\tilde{\mathbf{z}}^{(j+1)}$, the candidate ZOH controls are forward-integrated through the nonlinear dynamics with `ode45` to obtain the actual final mass m_f^{act} ;

- the LTV-predicted final mass is compared with this through the trust-region ratio $\eta = (m_f^{\text{act}} - m_f^{(j)}) / (m_f^{\text{pred}} - m_f^{(j)})$;
- if $\eta < \eta_l = 0.25$ the linear model is unreliable: the step is **rejected**, the reference is left unchanged, and $\rho \leftarrow \rho/2$;
- if $\eta_l \leq \eta < \eta_h = 0.7$ the step is accepted and ρ is held;
- if $\eta \geq \eta_h$ the linear model is excellent: the step is accepted and ρ is doubled (capped at the base value).

This is the canonical SCvx trust-region update [9, 8] and converges to the local optimum without the unphysical mass creation seen with a fixed unsafeguarded box.

The trust region is the cure for **artificial unboundedness**. A second failure mode, **artificial infeasibility**, arises when the linearised constraint set becomes empty even though the nonlinear problem is feasible. The canonical cure augments the linearised dynamics with unconstrained **virtual controls** $\boldsymbol{\nu}_k$ and relaxes the linearised path inequalities $\mathbf{g}$ with non-negative buffers $\mathbf{w}$,

$$\mathbf{x}_{k+1} = \bar{\mathbf{A}}_k \mathbf{x}_k + \bar{\mathbf{B}}_k \mathbf{u}_k + \bar{\mathbf{c}}_k + \boldsymbol{\nu}_k, \quad \mathbf{g}(\mathbf{x}_{\text{ref}}) + \frac{\partial \mathbf{g}}{\partial \mathbf{x}}(\mathbf{x} - \mathbf{x}_{\text{ref}}) + \mathbf{w} \leq \mathbf{0}, \quad (46)$$

both driven to zero by an exact-penalty term $J_\nu = \lambda_\nu \|\boldsymbol{\nu}\|_1 + \lambda_w \|\mathbf{w}\|_1$ added to the objective, whose weights are raised until the safeguards vanish at convergence [2, Sec. 3.4, Eqs. (3.37)–(3.40)]. So the trust region handles artificial unboundedness and the virtual controls handle artificial infeasibility. Infeasibility does show up briefly in my implementation: after the first four rejections shrink the trust region onto the trapezoidal reference (which is not exactly consistent with the ZOH dynamics), the inner solver reports the subproblem infeasible once, in both the **fmincon** and the **ECOS** variants. The loop survives without virtual controls only because the inner solver still returns its least-infeasible iterate, which the nonlinear replay ratio test then vets before accepting it as the new reference. A production implementation — and any cold-started run — should carry the virtual-control safeguard explicitly.

For the nonlinear powered-descent dynamics, the Jacobians evaluate to

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & -T_x/m^2 \\ 0 & 0 & 0 & 0 & -T_y/m^2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1/m & 0 \\ 0 & 1/m \\ -T_x/(c\|\mathbf{T}\|) & -T_y/(c\|\mathbf{T}\|) \end{bmatrix}. \quad (47)$$

The mass row of $\mathbf{B}$ is singular at $\|\mathbf{T}\| = 0$, so I regularise the denominator as $\sqrt{\|\mathbf{T}\|^2 + \epsilon^2}$ with $\epsilon^2 = 10^{-6}$ in non-dim units. The resulting $O(\epsilon)$ bias sits where $\|\mathbf{T}\| \sim \epsilon$, on the coast arc, which is also where the trust region struggles most. The

cleaner fix is the log-mass change of variables $z = \ln m$, under which $\dot{z} = -\|\tilde{\mathbf{T}}\|/c$ is linear and the singular row disappears (at the price of a thrust bound $\|\tilde{\mathbf{T}}\| \leq \tilde{T}_{\max} e^z$ that is linearised about z_{ref}) [2, Sec. 3.8.3].

Variant (c): SCvx with a conic inner solver (YALMIP+ECOS)

Variant (b) feeds the LTV sub-problem to `fmincon` as a generic NLP, which discards the convex structure of the inner step and pays the cost of a quasi-Newton SQP iteration. Variant (c) keeps the same outer SCvx loop (warm-start from the trapezoidal solution, identical adaptive trust-region update, identical convergence tolerance) but re-models the inner sub-problem as a Second-Order Cone Program. With LTV dynamics $(\bar{\mathbf{A}}_k, \bar{\mathbf{B}}_k, \bar{\mathbf{c}}_k)$ as linear equalities, glide-slope and box bounds as linear inequalities, and the thrust-magnitude bound as the second-order cone constraint $\|\mathbf{u}_k\|_2 \leq T_{\max}$, the inner step is a pure SOCP. The problem is assembled in YALMIP via `sdpvar` and solved by ECOS, which works on the conic structure directly. The two outer loops produce nearly indistinguishable trajectories (see Section 6.6), but the YALMIP/ECOS path is cleaner numerically: the inner subproblem keeps the cone constraint as a primitive instead of approximating it through SQP linearisations, so the per-step fidelity of the linearisation improves. The total wall time of the conic variant is about 29 s for the full 15-iteration outer loop, roughly three times faster than the `fmincon`-based variant (b) measured in the same session (Table 8). This is an end-to-end wall-time ratio on this problem and session, not a normalised per-iteration comparison: the conic solver treats each inner step as a native SOCP, whereas `fmincon` pays for a quasi-Newton SQP iteration with a finite-difference Jacobian (the dominant cost), so supplying analytic gradients to the `fmincon` path would partly close the gap.

6.3 Convex-Program Structure of the Inner Step

The inner sub-problem of variants (b) and (c) is a Second-Order Cone Program (SOCP), the convex class

$$\min_{\mathbf{x}} \mathbf{c}^T \mathbf{x} \quad \text{s.t.} \quad \mathbf{A}\mathbf{x} = \mathbf{b}, \quad \|\mathbf{F}_i \mathbf{x} + \mathbf{g}_i\|_2 \leq \mathbf{h}_i^T \mathbf{x} + r_i, \quad (48)$$

which sits at the top of the convex hierarchy $\text{LP} \subset \text{QP} \subset \text{QCQP} \subset \text{SOCP}$ [2, Sec. 3.2, Eqs. (3.21)–(3.24)]. Every constraint of the LTV step fits this template: the discrete dynamics $\mathbf{x}_{k+1} = \bar{\mathbf{A}}_k \mathbf{x}_k + \bar{\mathbf{B}}_k \mathbf{u}_k + \bar{\mathbf{c}}_k$ and the boundary conditions are linear equalities; the glide-slope half-spaces and box bounds are linear inequalities; and the only genuinely conic constraint is the thrust-magnitude bound $\|\mathbf{u}_k\|_2 \leq T_{\max}$, a single Lorentz cone per node. Being convex, the SOCP has a unique global optimum reachable in polynomial time by an interior-point method, which is what ECOS does in variant (c). Variant (b) reaches the same global optimum of the same convex

sub-problem, but it hands it to a general-purpose SQP that buries the cone inside successive quadratic approximations instead of working on it directly. In that sense variant (c) is the more natural way to write the inner step.

6.4 Lossless Convexification of the Thrust Constraint

The thrust bound in this homework is $0 \leq \|\mathbf{T}\| \leq T_{\max}$, with $T_{\min} = 0$; it is already convex (a Lorentz cone) and needs no special treatment. A non-zero lower bound $T_{\min} > 0$, by contrast, makes the annular set $T_{\min} \leq \|\mathbf{T}\| \leq T_{\max}$ non-convex. **Lossless convexification** restores convexity without changing the optimum: introduce a slack Γ and replace the magnitude constraint by the relaxed pair

$$\|\mathbf{T}\| \leq \Gamma, \quad T_{\min} \leq \Gamma \leq T_{\max}, \quad (49)$$

a second-order cone plus a box. The relaxation is provably exact—the cone is active ($\|\mathbf{T}\| = \Gamma$) at the optimum—a result established for the powered-descent problem by Açikmeşe and Ploen [7] and, for the free-final-time ascent problem, proved by a Pontryagin contradiction argument: an inactive cone would force $\lambda_m(t_f) = 0$, contradicting the transversality condition $\lambda_m(t_f) = 1$ [2, Prop. 1 and App. B]. With the conic SCvx infrastructure of variant (c) already in place, adding the slack (49) is the natural route to a $T_{\min} > 0$ extension, as noted in Section 7.

6.5 Forward-Integration Validation

The transcription accuracy of the four approaches (trapezoidal, ZOH–RK4, ZOH–LTV+SCvx with `fmincon`, ZOH–LTV+SCvx with `YALMIP+ECOS`) is assessed by forward-integrating the continuous-time dynamics (2)–(6) with `ode45` (RelTol = 10^{-10} , AbsTol = 10^{-12}) using the optimised control schedule, and comparing the resulting trajectory against the discretised values at every grid node. The control interpolation matches the transcription assumption: piecewise linear (PWL) for the trapezoidal rule, piecewise constant (PWC) for the three ZOH variants. The node error

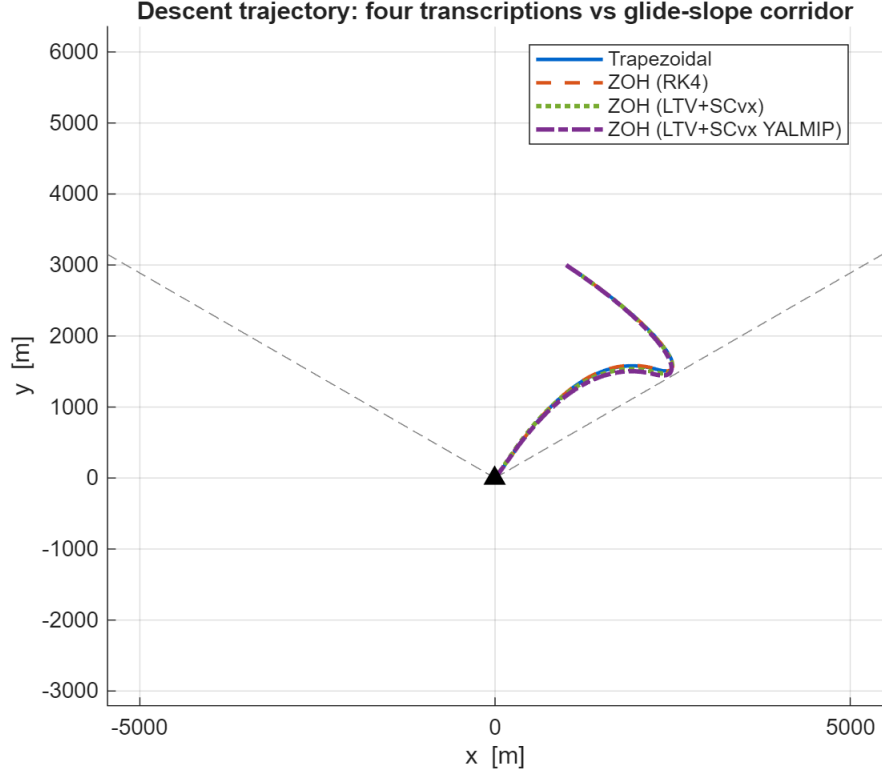
$$e_k = \|(\mathbf{r}, \mathbf{v})_k^{\text{NLP}} - (\mathbf{r}, \mathbf{v})^{\text{ode45}}(t_k)\|_2 \quad (50)$$

is the norm, in non-dim units, of the position and velocity components; the mass channel is monitored separately through the replayed final mass, which also drives the SCvx ratio test of Section 6.2. This metric quantifies how well each transcription represents the continuous-time problem.

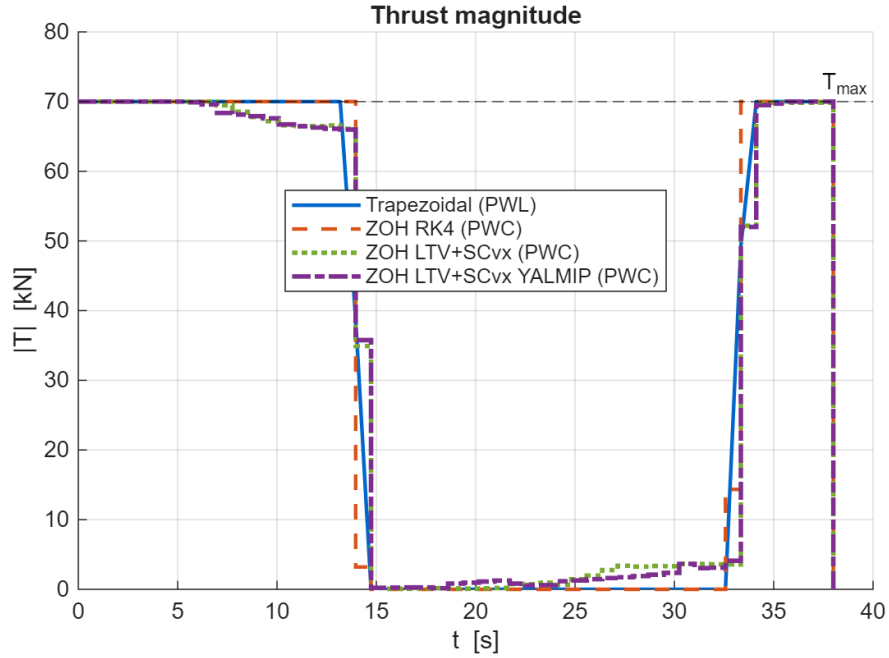
6.6 Results

The four transcriptions converge to qualitatively identical trajectories (Fig. 5a). All four respect the glide-slope corridor and softly land at the origin, with only sub-metric

differences in the divert phase. Figure 5b shows the thrust profiles: the trapezoidal thrust is piecewise linear, whereas the three ZOH variants are naturally piecewise constant (rendered as staircases). All four exhibit the max-coast-max structure, with $\|\mathbf{T}\|$ saturated at $T_{\max}$ on the two burn arcs and at zero in between.

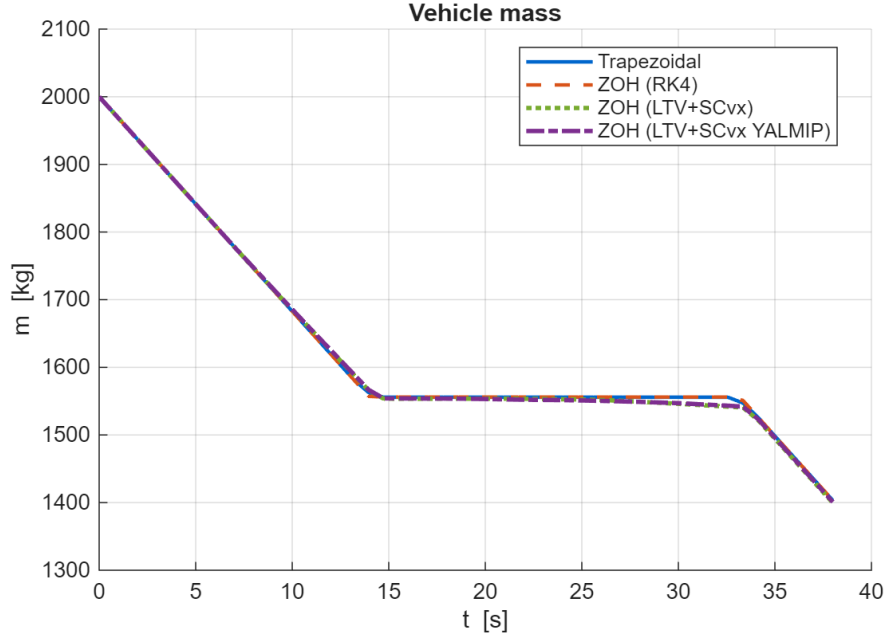


(a) Trajectory in the (x, y) plane.

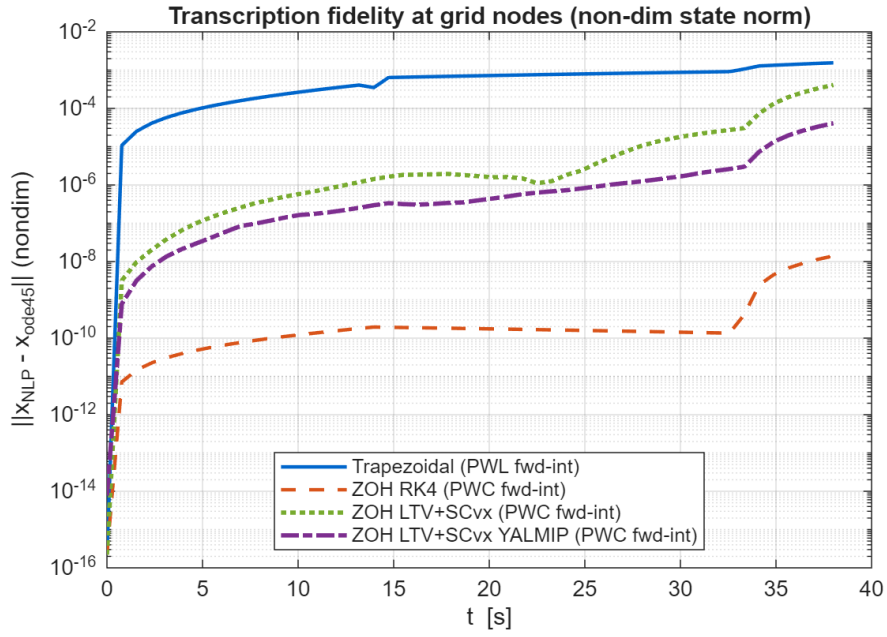


(b) Thrust magnitude: PWL (trapezoidal) vs PWC (ZOH).

Figure 5: Comparison of the four transcriptions at $t_f = 38$ s.



(a) Vehicle mass profile.



(b) Forward-integration error at grid nodes (log scale).

Figure 6: Mass profile and transcription-fidelity check via `ode45` forward integration.

The transcription-fidelity plot (Fig. 6b) is the main diagnostic of Task 2: it measures how far each discretised trajectory drifts from a high-fidelity `ode45` integration of the same controls. Table 7 gives a representative numerical comparison.

Table 7: Four-way comparison at $t_f = 38$ s, $N = 50$ nodes.

Transcription	m_f [kg]	Fuel [kg]	Order	Max node error [-]
Trapezoidal	1403.20	596.80	$O(\Delta t^2)$	$1.55 \cdot 10^{-3}$
Nonlinear ZOH (RK4)	1403.37	596.63	$O(\Delta t^4)$	$1.40 \cdot 10^{-8}$
LTV ZOH + SCvx (fmincon)	1399.54	600.46	ode45 -based	$4.08 \cdot 10^{-4}$
LTV ZOH + SCvx (YALMIP)	1400.84	599.16	ode45 -based	$4.09 \cdot 10^{-5}$

The trapezoidal transcription has a global $O(\Delta t^2)$ error and sits at $\sim 10^{-3}$ in non-dim state norm (a few metres of position, fractions of a m/s of velocity). The nonlinear-ZOH+RK4 transcription is $O(\Delta t^4)$ and drops to the **ode45** tolerance floor ($\sim 10^{-8}$). Both LTV+SCvx variants lie in between: they propagate the linearised system with **ode45** on each interval, so the per-step accuracy is high, but the converged solution is constrained by the (small) residual linearisation error within the final trust radius. The conic variant (c) is one order of magnitude cleaner than the **fmincon** variant (b) ($4 \cdot 10^{-5}$ vs $4 \cdot 10^{-4}$): exposing the second-order cone constraint to the inner solver instead of approximating it by SQP linearisations measurably improves the per-step fidelity.

The trapezoidal and nonlinear-ZOH transcriptions agree to within 0.2 kg on m_f , indicating both have located the same local optimum. The two SCvx solutions sit 2–4 kg below this (slightly more fuel burnt) and agree with each other to within 1.3 kg. Two contributing effects are at play:

- Trapezoidal collocation enforces the dynamics only to second order in Δt , so its converged “optimum” is allowed a small dynamical residual ($\sim 10^{-3}$ above) that translates into a marginally optimistic final mass.
- Both SCvx outer loops stop at the 15-iteration cap with $\|\Delta \mathbf{x}\| \approx 10^{-2}$, above the convergence tolerance of 10^{-3} . Every accepted step, however, has passed the ratio test against an **ode45** replay of the nonlinear dynamics, so the returned iterate is realisable; the residual gap just means the loop settles in a slightly different but nearby local optimum.

Both effects are below the engineering accuracy of the model (no aerodynamics, no attitude). Within numerical noise the four transcriptions describe the same physical solution.

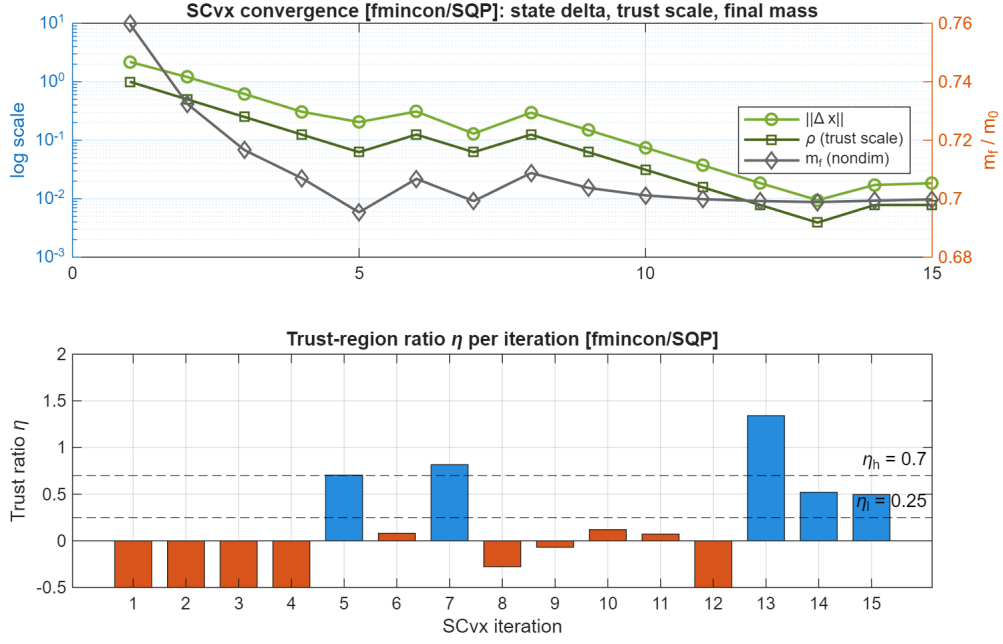
The same **ode45** replay also gives the most operationally useful figure of merit: the touchdown dispersion obtained by flying each optimised control schedule through the true continuous-time dynamics. Table 8 reports the resulting position and velocity error norms at t_f (target: the origin at rest), the drift of the replayed final mass with respect to the transcription’s own m_f , and the wall time of each solve.

Table 8: Replay touchdown dispersion (SI units) and wall time, $t_f = 38$ s, $N = 50$.

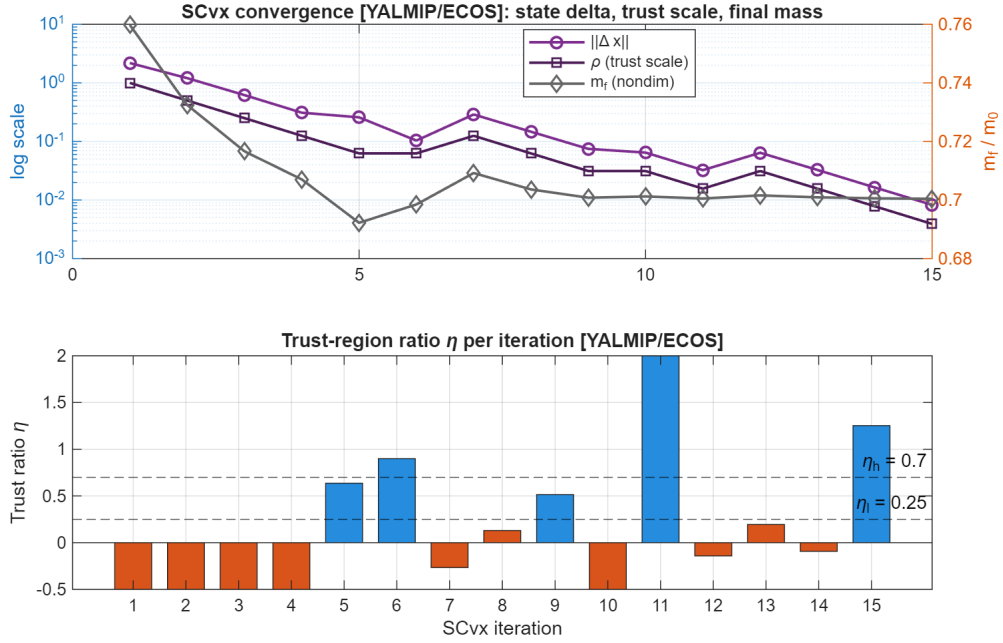
Transcription	Pos. err [m]	Vel. err [m/s]	Δm_f [kg]	Wall [s]
Trapezoidal (PWL)	4.3	0.11	-0.03	90
Nonlinear ZOH (RK4)	$6.1 \cdot 10^{-6}$	$2.4 \cdot 10^{-6}$	$1.6 \cdot 10^{-5}$	100
LTV ZOH + SCvx (fmincon)	0.18	0.069	0.45	82
LTV ZOH + SCvx (YALMIP)	0.019	0.0069	0.045	29

Flying the trapezoidal PWL controls open-loop lands the vehicle 4.3 m from the pad at 0.11 m/s, which is fine for an offline reference trajectory and matches the dispersion expected from its $\sim 10^{-3}$ transcription error. The ZOH–RK4 schedule replays to micrometre accuracy (its defects are the RK4 flow map), while the two SCvx variants land within centimetres to decimetres. All wall times are measured in the same batch session on the same laptop; the conic inner solver makes the SCvx loop about three times faster than every **fmincon**-based solve.

The SCvx convergence traces are reported in Fig. 7. For each variant the top panel overlays the iterate change $\|\mathbf{x}^{(j+1)} - \mathbf{x}^{(j)}\|_2$, the trust-region scale ρ , and the final mass $m_f^{(j)}$; the bottom panel shows the trust-region ratio η against its acceptance band $[\eta_l, \eta_h] = [0.25, 0.7]$ (rejected steps are flagged in orange). Warm-started from the Task 1 trapezoidal solution, both algorithms initially overshoot the linearisation’s region of validity and reject the first four candidate steps, shrinking ρ ; from then on accepted and rejected steps alternate as the ratio test probes the trust radius, and $\|\Delta \mathbf{x}\|$ settles at the 10^{-2} level by the iteration cap. The final mass m_f stabilises within a few kilograms of the trapezoidal and nonlinear-ZOH values.



(a) Variant (b): SCvx with fmincon/SQP inner solver.



(b) Variant (c): SCvx with YALMIP+ECOS inner solver.

Figure 7: SCvx convergence traces for the two LTV+SCvx variants. Top panels: $\|\mathbf{x}^{(j+1)} - \mathbf{x}^{(j)}\|_2$ (left axis, log), trust scale ρ , and final mass m_f (right axis) per outer iteration. Bottom panels: trust-region ratio η against acceptance band $[\eta_l, \eta_h] = [0.25, 0.7]$, with rejected steps highlighted in orange.

The final-mass values returned by the four transcriptions agree to within a few kilograms, which confirms that the trapezoidal approximation is accurate enough for engineering purposes despite its lower-order convergence. The ZOH-RK4 variant is the simplest implementation when all I need is a high-fidelity transcription; the two LTV+SCvx variants expose the convex-sub-problem structure that underpins

production guidance algorithms (GFOLD, lossless convexification, and so on). The conic variant (c) writes the inner step as an SOCP and matches variant (b) in trajectory and final mass while giving a cleaner per-step fidelity.

7 Conclusions

I transcribed the minimum-fuel powered-descent and pinpoint-landing optimal control problem into a finite-dimensional NLP via trapezoidal direct collocation, and solved it with `fmincon`/SQP. The main findings are below.

Direct vs. indirect transcription. Compared to the indirect-shooting approach of Homework 1, direct collocation requires no analytical derivation of costate equations or transversality conditions, accepts path constraints (glide-slope, thrust bounds) directly as inequality constraints, and tolerates a coarse initial guess [1, Sec. 4.12.7]. The cost is a higher-dimensional NLP (350 variables here) and the loss of the analytical insight that the bilinear-tangent law gave in the indirect formulation.

Max-coast-max thrust structure. The fuel-optimal solution shows the three-arc bang-off-bang behaviour expected for problems with linear-in-control dynamics and a control-magnitude bound: a saturated braking burn at $T_{\max}$, a midcourse coast arc at $T_{\min} = 0$, and a second saturated burn down to touchdown. This is the direct-transcription analogue of the maximum-principle prediction (Section 2.5): the optimal thrust magnitude never sits at intermediate values.

Sensitivity to flight time. A $\pm 5\%$ sweep around the nominal $t_f = 38$ s showed fuel consumption growing monotonically with flight time (a longer horizon stretches the coast arc and increases the cumulative gravity loss), with a $\approx 1.3\%$ propellant variation across the window. The lower bound on t_f is set implicitly by the thrust-magnitude constraint.

Active constraints at the optimum. The only path inequality active at the optimum is the upper thrust bound $\|\mathbf{T}\| = T_{\max}$, binding on the two burn arcs with strictly positive KKT multipliers. The glide-slope cone stays inactive: the trajectory approaches the corridor boundary near the apex of the divert turn with a minimum margin of 1° – 2° across the sensitivity sweep, and the corresponding multipliers are numerically zero. By complementary slackness, the thrust bound is the only binding constraint shaping the trajectory.

ZOH transcription (Task 2). The optional ZOH discretisation of Section 6 was implemented in three forms—a nonlinear ZOH with RK4 propagation, the literal Appendix A construction (LTV-linearised + SCvx) solved by `fmincon`/SQP, and the same SCvx outer loop with the inner sub-problem cast as a Second-Order Cone Program and solved by YALMIP+ECOS. All three match the trapezoidal baseline at the engineering level: final masses agree within a few kilograms ($\lesssim 0.7\%$ of the propellant). The forward-integration test exposes the lower convergence order of the trapezoidal rule, with the ZOH–RK4 variant dropping the per-node fidelity error by five orders of magnitude and sitting close to the `ode45` tolerance floor. Warm-started from the trapezoidal solution and stabilised by an adaptive trust-region ratio test, both SCvx variants settle on the same nearby local optimum. The conic variant exposes the SOCP

structure of the inner step directly and gives a per-step fidelity an order of magnitude tighter than the `fmincon`/SQP path, which is the convex-sub-problem framework behind production guidance algorithms (GFOLD, lossless convexification) [7, 10].

Non-dimensionalisation. Following the approach used in Homework 1, the OCP is solved in non-dimensional variables (Section 3.1), with $L_{ref} = y_0$, $a_{ref} = g$ and the single residual parameter $V_c = V_{ref}/c \approx 0.078$. This shrinks the dynamic range of the decision vector from four orders of magnitude to one and is needed for the LTV linearisation to be well-conditioned: the Jacobians become uniformly $O(1)$ and `fmincon`/SQP makes real progress per iteration.

Limitations and roadmap. The model neglects aerodynamic drag, vehicle attitude dynamics, and any throttling rate limits, and the flight time is fixed. Extensions left beyond the scope of this report are:

- A **free-time variant** that lifts the fixed- t_f assumption and allows the solver to find the absolute fuel-optimal flight time subject to the active thrust constraint, e.g. as in lossless-convexification formulations.
- **Lossless convexification** of a non-zero lower thrust bound $T_{min} > 0$, which is non-convex in $\mathbf{T}$: with the conic SCvx infrastructure already in place (variant (c)), the next step is to add the slack-variable trick of Açikmeşe and Ploen [7] to recover a convex SOCP in the new state, and check that the original constraint is recovered at the optimum (Section 6.4).
- The **virtual-control safeguard** against artificial infeasibility in the SCvx inner problem, which surfaced transiently in Section 6: the penalised buffer ν_k would make the outer loop robust to cold starts.
- Inclusion of **aerodynamic drag** and **attitude dynamics**, moving from a 3-DoF point mass to a 6-DoF rigid body with angle-of-attack and angle-of-sideslip dependence. Once the thrust is optimised, the aerodynamic loads also need a dynamic-pressure constraint $q = \frac{1}{2}\rho v_{rel}^2 \leq q_{max}$ (enforced as a hard path constraint, or checked a posteriori when the thrust profile is prescribed)—the structural max- q load case that Homework 3 deals with [2, Sec. 4.4.8].

A Implementation: Key Code Excerpts

The full MATLAB sources (`main_task1.m`, `main_task2.m`, `ode_descent.m`, `rk4_zoh.m`) live in the `HM2_powered_descent/` folder of the repository. This appendix collects the three routines that carry the two transcriptions: the non-dimensional dynamics, the trapezoidal collocation constraints, and the RK4 zero-order-hold propagator used in Task 2. Everything runs on `fmincon` alone (no CasADi, no YALMIP) except the conic SCvx variant of Task 2.

Both transcriptions and the test suite call the same right-hand side in Listing 1, which evaluates the non-dimensional dynamics (2)–(6) with gravity normalized to 1.

```

1 function dx = ode_descent(x, u, Vc)
2 % Non-dimensional powered-descent point-mass dynamics (HM2).
3 %   x = [x; y; vx; vy; m]   non-dim state
4 %   u = [Tx; Ty]             non-dim thrust components
5 %   Vc = V_ref / c           effective Tsiolkovsky number
6 %
7 % Returns dx = d/dt [x; y; vx; vy; m] with non-dim gravity = -1.
8 % Shared by main_task1.m, main_task2.m and the test suite.
9 %
10 % No arguments validation by design: hot-loop RHS called by ode45 and the
11 % fmincon transcriptions; validate at the call site.
12
13 Tmag = sqrt(u(1)^2 + u(2)^2);
14 dx = [ x(3); x(4); u(1)/x(5); u(2)/x(5) - 1; -Vc * Tmag ];
15
16 end

```

Listing 1: Non-dimensional point-mass descent dynamics (`ode_descent.m`).

Listing 2 is the nonlinear-constraint routine of the trapezoidal NLP. It builds the $5(N-1)$ defect equalities of Eq. (17) from the node values of the right-hand side, then stacks the $4N$ thrust-magnitude and glide-slope inequalities described in Section 3.

```

258 function [c_ineq, c_eq] = trap_nonlcon(z, N, dt, d)
259     Z = reshape(z, 7, N);
260
261     % --- Dynamics RHS at every node (non-dim) ---
262     f = zeros(5, N);
263     for i = 1:N
264         f(:,i) = dyn_rhs(Z(:,i), d.Vc);
265     end
266
267     % --- Trapezoidal defects ---
268     defs = zeros(5, N-1);
269     for k = 1:N-1
270         defs(:,k) = Z(1:5, k+1) - Z(1:5, k) - 0.5*dt*(f(:,k) + f(:,k+1));
271     end
272     c_eq = defs(:);
273
274     % --- Path constraints ---
275     Tmag = sqrt(Z(6,:).^2 + Z(7,:).^2).';
276     g_thr_lo = d.Tmin - Tmag;
277     g_thr_hi = Tmag - d.Tmax;
278     tt = tan(d.theta_mx);
279     g_gs_pos = ( Z(1,:).' - tt*Z(2,:).' );

```

```

280     g_gs_neg = (-Z(1,:).' - tt*Z(2,:).');
281
282     c_ineq = [g_thr_lo; g_thr_hi; g_gs_pos; g_gs_neg];
283 end

```

Listing 2: Trapezoidal defects and path constraints (`trap_nonlcon`, `main_task1.m`).

The Task 2 nonlinear ZOH transcription holds the thrust constant over each interval and propagates the dynamics with a fixed-step RK4 scheme; this is the propagator in Listing 3.

```

1 function x_next = rk4_zoh(x, u, dt, Vc, n_sub)
2 % Propagate the non-dim descent dynamics (ode_descent) over one ZOH
3 % interval of length dt with a fixed-step RK4 scheme, holding the control
4 % u constant, using n_sub substeps.
5 % Shared by main_task2.m (nonlinear ZOH transcription) and the test suite.
6 %
7 % No arguments validation by design: hot-loop propagator called by the
8 % fmincon ZOH transcription; validate at the call site.
9
10 h = dt / n_sub;
11 for ii = 1:n_sub
12     k1 = ode_descent(x, u, Vc);
13     k2 = ode_descent(x + 0.5*h*k1, u, Vc);
14     k3 = ode_descent(x + 0.5*h*k2, u, Vc);
15     k4 = ode_descent(x + h*k3, u, Vc);
16     x = x + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
17 end
18 x_next = x;
19
20 end

```

Listing 3: RK4 zero-order-hold propagator (`rk4_zoh.m`).

References

- [1] J. T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. SIAM, Philadelphia, 2nd edition, 2010.
- [2] B. Benedikter. *Convex Optimization of Launch Vehicle Ascent Trajectories*. PhD thesis, Sapienza University of Rome, Rome, Italy, 2022. Ph.D. in Aeronautical and Space Engineering, XXXIV cycle.
- [3] C. R. Hargraves and S. W. Paris. Direct trajectory optimization using nonlinear programming and collocation. *Journal of Guidance, Control, and Dynamics*, 10(4):338–342, 1987.
- [4] A. Zavoli. Dynamics and control of launch vehicles — course lecture notes. Graduate course, Sapienza University of Rome, 2026. Academic Year 2025/26.
- [5] J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.
- [6] A. E. Bryson and Y.-C. Ho. *Applied Optimal Control: Optimization, Estimation and Control*. Taylor & Francis, New York, revised edition, 1975.
- [7] B. Açıkmeşe and S. R. Ploen. Convex programming approach to powered descent guidance for mars landing. *Journal of Guidance, Control, and Dynamics*, 30(5):1353–1366, 2007.
- [8] D. Malyuta, T. P. Reynolds, M. Szmuk, T. Lew, R. Bonalli, M. Pavone, and B. Açıkmeşe. Convex optimization for trajectory generation: A tutorial on generating dynamically feasible trajectories reliably and efficiently. *IEEE Control Systems Magazine*, 42(5):40–113, 2022.
- [9] Y. Mao, M. Szmuk, and B. Açıkmeşe. Successive convexification of non-convex optimal control problems and its convergence properties. In *Proceedings of the 55th IEEE Conference on Decision and Control (CDC)*, pages 3636–3641, 2016.
- [10] L. Blackmore, B. Açıkmeşe, and D. P. Scharf. Minimum-landing-error powered-descent guidance for Mars landing using convex optimization. *Journal of Guidance, Control, and Dynamics*, 33(4):1161–1171, 2010.